

Retro Arena Survivor

Group Number: 5

White tiger Team Members

2411310770 สิริคนัย ศิริมงคลมิตร

2411310572 รัชชานนท์ ขวัญแก้ว

2411312073 พฤกษ์ จันทรสวาท

Overview

- แนวคิดโปรเจกต์: สร้างเกมตู้ Arcade ยุค Retro 8-bit แนว Action-Survival บนบอร์ดประมวลผล ESP32
- จุดเด่นของระบบ:
 - แสดงผลกราฟิกแบบ Real-time บนจอ TFT LCD (3.8 inch)
 - ควบคุมทิศทางด้วย Analog Joystick และการปรับแสงจอด้วย Potentiometer ผ่านสัญญาณ PWM
 - มีระบบไฟ Status LEDs แสดง HP ภายนอก และเสียงเอฟเฟกต์ SFX ผ่าน Passive Buzzer
 - ประมวลผลด้วยสถาปัตยกรรม Non-blocking Logic (FSM) ทำให้ระบบลื่นไหล ไม่ค้าง

detail/requirement

ความต้องการทางด้านซอฟต์แวร์ (Software Requirements):

ตัวเกมต้องรันได้ไม่ต่ำกว่า 30 FPS โดยไม่ใช้ฟังก์ชัน `delay()` เพื่อลด Latency

รองรับระบบ Finite State Machine (FSM) 4 สถานะ: TITLE, PLAYING, LEVELUP, GAMEOVER

ความต้องการทางด้านฮาร์ดแวร์ (Hardware Requirements):

มีวงจรตรวจจับสัญญาณ Joystick อนุล็อก พร้อมระบบกำจัดสัญญาณรบกวน (Deadband Filtering)

มีอินเทอร์เฟซตอบสนองผู้ใช้ (Feedback) ครบทั้งภาพ (TFT/LED) และเสียง (Buzzer)

การจัดสรรขาพินและฮาร์ดแวร์ไอโอ (I/O Distribution)

GPIO Pin	โหมดการทำงาน	อุปกรณ์ฮาร์ดแวร์	หน้าที่หลักในตรรกะเกม
GPIO 34	ADC1	Analog Joystick X	ควบคุมการเคลื่อนที่แนวแกน X และเลื่อนเมนู
GPIO 33	ADC1	Analog Joystick Y	ควบคุมการเคลื่อนที่แนวแกน Y และเลือกเมนูอัปเกรด
GPIO 25	PULLUP	Tactile Push Button	ปล่อยสกิลระเบิดลูกเงิน (AoE) และกดตกลงเมนู
GPIO 32	ADC1	Contrast Potentiometer	ปรับความสว่างไฟ Backlight และปิดจอดำเมื่อต่ำสุด
SPI Pins	TFT_eSPI	3.8" TFT LCD Display	แสดงผลกราฟิก HUD, XP, ตัวละคร และศัตรูเรียลไทม์
GPIO 16, 17, 5	OUTPUT	Tri-Color LED Array	แสดงไฟสถานะ HP 3 ระดับ (เขียว/เหลือง/แดง)
GPIO 26	PWM/Tone	Passive Piezo Buzzer	สังเคราะห์เสียงเอฟเฟกต์ SFX, เพชร, การชน และ Fanfare

specification

Microcontroller: ESP32 DevKit (32-bit Dual-Core, 240MHz, 520KB SRAM)

Display Output: จอ TFT LCD 3.8" (ILI9488/ST7796S) ผ่าน SPI

Analog Inputs:

2-Axis Analog Joystick (ADC Pins: GPIO 34, 33 / Deadband range: 1400–2600)

Potentiometer 10k Ω (ADC Pin: GPIO 32) ปรับ Backlight (PWM Pin: GPIO 14)

Digital Inputs & Outputs:

Push Button (Internal Pull-up: GPIO 25) สกิลระเบิด / ยืนยันเมนู

Status LEDs 3 สี (GPIO 16=เขียว, 17=เหลือง, 5=แดง) แสดงสถานะ HP

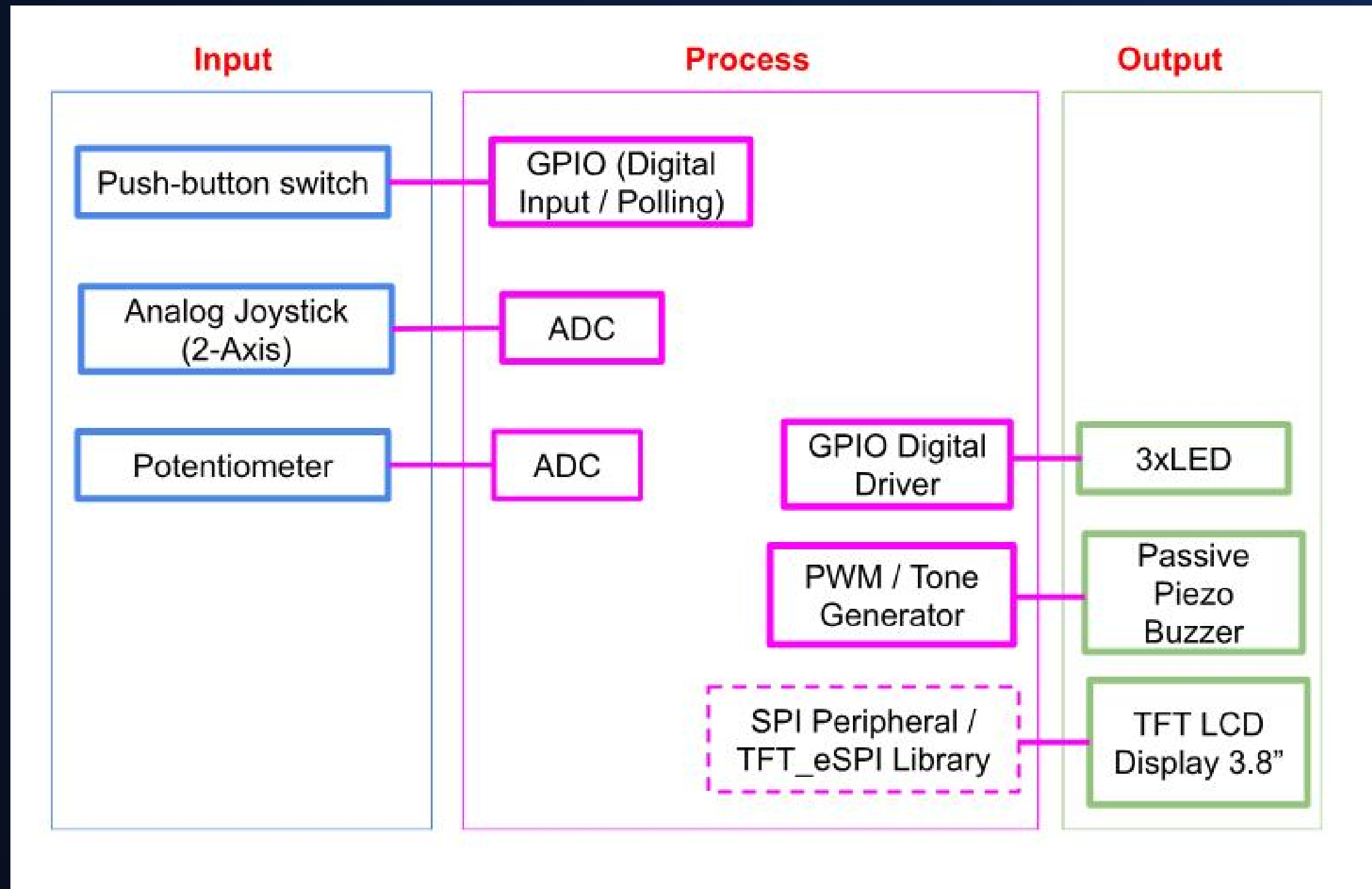
Passive Piezo Buzzer (GPIO 26) กำเนิดความถี่เสียง SFX

Project plan (and actual)

สัปดาห์ที่ (Week)	กิจกรรมตามแผน (Planned Task)	ผลการทำงานจริง (Actual Result)	สถานะ (Status)
Week 1-2	ศึกษารายละเอียด โจทย์ และออกแบบสถาปัตยกรรมระบบ	สำเร็จตามแผน	Completed
Week 3-4	ต่อวงจรฮาร์ดแวร์บน Breadboard และทดสอบเซนเซอร์	สำเร็จตามแผน (มีการปรับเปลี่ยนขา GPIO)	Completed
Week 5-6	พัฒนาโค้ด FSM, Non-blocking Logic และระบบเกม	สำเร็จตามแผน	Completed
Week 7-8	ประกอบชิ้นงานจริง, ทดสอบระบบรวม (Integration) และจัดทำเอกสาร	สำเร็จตามแผน	Completed

Architectural design & hardware selection

System block diagram



Input (ฝั่งซ้าย): ครอบคลุมทั้ง Push-button switch, Analog Joystick (2-Axis) และ Potentiometer

Process (กลาง): ระบุส่วนประมวลผลภายในชิปชัดเจน ทั้ง GPIO (Digital Input / Polling), ADC, GPIO Digital Driver, PWM / Tone Generator และ SPI Peripheral / TFT_eSPI Library

Output (ฝั่งขวา): ครอบคลุมทั้ง 3xLED, Passive Piezo Buzzer และ TFT LCD Display (3.8")

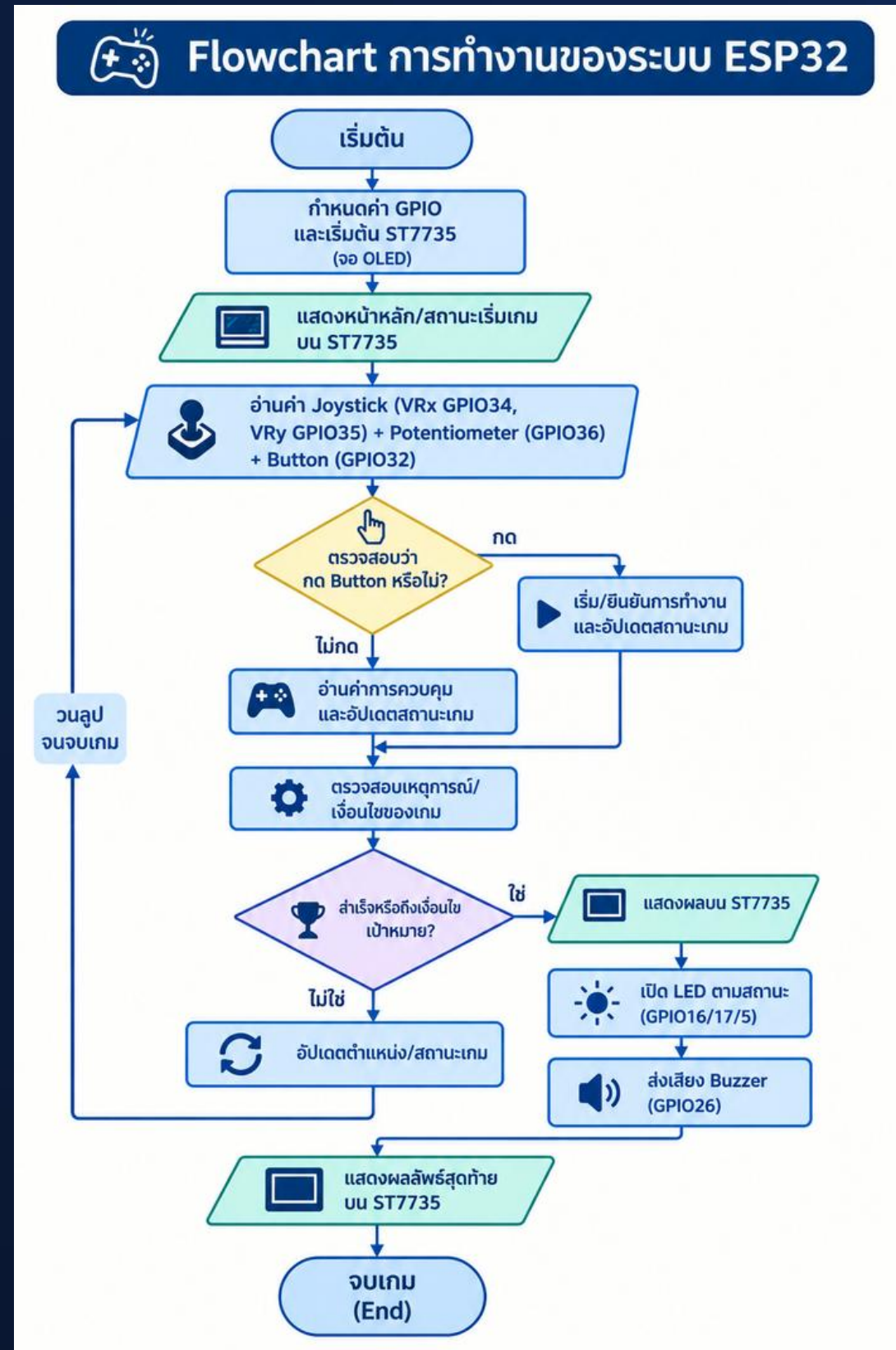
เหตุผลในการเลือกฮาร์ดแวร์ :

ESP32: มีความเร็ว Clock สูง และมีพอร์ต ADC/PWM เพียงพอต่อการใช้งาน

Passive Buzzer: รองรับการสร้างโทนเสียงความถี่หลากหลายผ่านฟังก์ชัน `tone()` ต่างจาก **Active Buzzer** ที่สร้างได้เสียงเดียว

Detailed design

Flowchart



Core System & FSM Control Module

```
617 // --- SETUP & LOOP ---
618 void setup() {
619     Serial.begin(115200);
620
621     pinMode(PIN_SKILL_BTN, INPUT_PULLUP);
622     pinMode(PIN_JOY_X, INPUT);
623     pinMode(PIN_JOY_Y, INPUT);
624     pinMode(PIN_POT_BRIGHT, INPUT);
625
626     pinMode(PIN_LED_GRN, OUTPUT);
627     pinMode(PIN_LED_YEL, OUTPUT);
628     pinMode(PIN_LED_RED, OUTPUT);
629
630     pinMode(PIN_BUZZER, OUTPUT);
631     digitalWrite(PIN_BUZZER, LOW);
632
633     initBacklight();
634
635     tft.init();
636     tft.setRotation(2); // Inverted portrait orientation matching your breadboard
637     tft.fillScreen(TFT_BLACK);
638
639     stateEnterTime = millis();
640     titleScreenDrawn = false;
641     currentState = STATE_TITLE;
642 }
```

```
644 void loop() {
645     updateBuzzer();
646
647     int potVal = analogRead(PIN_POT_BRIGHT);
648     if (potVal < 150) {
649         setBacklight(0);
650     } else {
651         uint8_t duty = map(potVal, 150, 4095, 20, 255);
652         setBacklight(duty);
653     }
654
655     switch (currentState) {
656     case STATE_TITLE:
657         updateTitleScreen();
658         break;
659     case STATE_PLAYING:
660         updatePlayingScreen();
661         break;
662     case STATE_LEVELUP:
663         updateLevelUpScreen();
664         break;
665     case STATE_GAMEOVER:
666         updateGameOverScreen();
667         break;
668     }
669
670     delay(12);
671 }
```

setup() & loop():

- **setup()** กำหนดโหมดขา GPIO (Input/Output/Pull-up), เริ่มต้นบอร์ดประมวลผล, อินเทอร์เฟซ SPI ของหน้าจอ TFT, ระบบ PWM ขับ Backlight และตั้งค่าเข้าสู่หน้าจอเริ่มเกม (STATE_TITLE)
- **loop()** รันรูปประมวลผลหลักแบบ Non-blocking โดยใช้วงจร switch (currentState) เพื่อสลับการทำงานตามสถานะเกมในสถาปัตยกรรม FSM ได้แก่ TITLE, PLAYING, LEVELUP, และ GAMEOVER

Input Signal Processing Module (การจัดการสัญญาณอินพุต)

```
// --- STATE: PLAYING SCREEN ---
void updatePlayingScreen() {
    survivalTimeSec = (millis() - gameStartTime) / 1000;

    oldPlayerX = playerX;
    oldPlayerY = playerY;

    // 1. Controls with inverted axes
    int rawX = analogRead(PIN_JOY_X);
    int rawY = analogRead(PIN_JOY_Y);

    if (rawY > 2600 && playerX > 4) playerX -= 2.8;           // Left
    if (rawY < 1400 && playerX < SCREEN_W - PLAYER_SIZE - 4) playerX += 2.8; // Right
    if (rawX > 2600 && playerY < SCREEN_H - PLAYER_SIZE - 4) playerY += 2.8; // Down
    if (rawX < 1400 && playerY > 26) playerY -= 2.8;          // Up
}
```

```
296 void updateLevelUpScreen() {
297     int rawX = analogRead(PIN_JOY_X); // Vertical axis (Up/Down)
298
299     if (millis() - joyMenuCooldown > 250) {
300         if (rawX < 1400) {
301             if (menuSelection != 0) {
302                 menuSelection = 0;
303                 playSelectSound();
304                 joyMenuCooldown = millis();
305             }
306         } else if (rawX > 2600) {
307             if (menuSelection != 1) {
308                 menuSelection = 1;
309                 playSelectSound();
310                 joyMenuCooldown = millis();
311             }
312         }
313     }
314 }
```

```
111 void initBacklight() {
112     #if ESP_ARDUINO_VERSION >= ESP_ARDUINO_VERSION_VAL(3, 0, 0)
113         ledcAttach(TFT_LED_PIN, pwmFreq, pwmResolution);
114     #else
115         ledcSetup(0, pwmFreq, pwmResolution);
116         ledcAttachPin(TFT_LED_PIN, 0);
117     #endif
118     setBacklight(255);
119 }
```

initBacklight() & Potentiometer Dimmer:

อ่านค่าอนาล็อกจาก Potentiometer (GPIO 32) แล้วแปลงค่าด้วย map() เป็นสัญญาณ PWM (0-255) ส่งผ่าน setBacklight() ขับไฟ Backlight (GPIO 14) แบบ Real-time

Joystick Movement & Menu Navigation:

- อ่านค่า ADC1 ผ่าน analogRead(PIN_JOY_X) และ analogRead(PIN_JOY_Y) โดยตรงภายใน updatePlayingScreen() และ updateLevelUpScreen()
- กรองสัญญาณรบกวนด้วย Deadband Filtering (ช่วงค่า 1400-2600) เพื่อป้องกันตัวละครเคลื่อนที่เองเมื่อจอยอยู่นิ่ง

Game Logic & Mechanics Module (ส่วนคำนวณเกมและสกิล)

```
366 // --- STATE: PLAYING SCREEN ---
367 void updatePlayingScreen() {
368     survivalTimeSec = (millis() - gameStartTime) / 1000;
369
370     oldPlayerX = playerX;
371     oldPlayerY = playerY;
372
373     // 1. Controls with inverted axes
374     int rawX = analogRead(PIN_JOY_X);
375     int rawY = analogRead(PIN_JOY_Y);
376
377     if (rawY > 2600 && playerX > 4) playerX -= 2.8; // Left
378     if (rawY < 1400 && playerX < SCREEN_W - PLAYER_SIZE - 4) playerX += 2.8; // Right
379     if (rawX > 2600 && playerY < SCREEN_H - PLAYER_SIZE - 4) playerY += 2.8; // Down
380     if (rawX < 1400 && playerY > 26) playerY -= 2.8; // Up
381 }
```

1. Controls with inverted axes (ส่วนการควบคุมและเคลื่อนที่)

หน้าที่: อ่านค่าอนาล็อกจาก Joystick (แกน X และ Y) เพื่อคำนวณและอัปเดตพิกัดตำแหน่งของผู้เล่น (playerX, playerY) พร้อมสลับทิศทางให้ถูกต้องตามการวางจอ และกำหนดขอบเขตไม่ให้ตัวละครเดินหลุดออกนอกจอ

```
382 // 2. Whip attack visual & cleanup
383 if (whipVisualActive && (millis() - whipVisualTimer > 120)) {
384     whipVisualActive = false;
385     tft.drawCircle(whipCenterX, whipCenterY, activeWhipRadius, TFT_BLACK);
386 }
387
388 if (millis() - lastAutoAttackTime >= autoAttackInterval) {
389     lastAutoAttackTime = millis();
390     whipVisualActive = true;
391     whipVisualTimer = millis();
392     whipCenterX = (int)playerX + (PLAYER_SIZE / 2);
393     whipCenterY = (int)playerY + (PLAYER_SIZE / 2);
394     activeWhipRadius = whipRadius;
395
396     tft.drawCircle(whipCenterX, whipCenterY, activeWhipRadius, TFT_YELLOW);
397
398     for (int i = 0; i < MAX_ENEMIES; i++) {
399         if (enemies[i].active) {
400             float d = sqrt(pow(enemies[i].x - playerX, 2) + pow(enemies[i].y - playerY, 2));
401             if (d <= whipRadius) {
402                 enemies[i].active = false;
403                 tft.fillRect((int)enemies[i].x, (int)enemies[i].y, ENEMY_SIZE, ENEMY_SIZE, TFT_BLACK);
404                 killCount++;
405                 spawnGem(enemies[i].x, enemies[i].y);
406                 playKillSound();
407             }
408         }
409     }
410 }
```

2. Whip attack visual & cleanup (ส่วนการโจมตีด้วยแส้อัตโนมัติ)

หน้าที่: จัดการระบบการโจมตีหลักของผู้เล่น โดยคำนวณระยะการโจมตีแบบวงกลม (Whip Radius) รอบตัวผู้เล่นตามช่วงเวลา (autoAttackInterval) หากศัตรูอยู่ในระยะจะถูกกำจัด เพิ่มคะแนน/สร้าง Gem และจัดการเคลียร์เส้นวงกลมกราฟิกบนหน้าจอเมื่อแสดงผลครบเวลา

Game Logic & Mechanics Module (ส่วนคำนวณเกมและสกิล)

```
412 // 3. Emergency Bomb skill visual & cleanup
413 if (bombVisualActive && (millis() - bombVisualTimer > 250)) {
414     bombVisualActive = false;
415     tft.drawCircle(bombCenterX, bombCenterY, BOMB_RADIUS, TFT_BLACK);
416     tft.drawCircle(bombCenterX, bombCenterY, BOMB_RADIUS - 10, TFT_BLACK);
417 }
418
419 if (digitalRead(PIN_SKILL_BTN) == LOW && (millis() - lastBombTime >= BOMB_COOLDOWN)) {
420     lastBombTime = millis();
421     bombVisualActive = true;
422     bombVisualTimer = millis();
423     bombCenterX = (int)playerX + (PLAYER_SIZE / 2);
424     bombCenterY = (int)playerY + (PLAYER_SIZE / 2);
425
426     tft.drawCircle(bombCenterX, bombCenterY, BOMB_RADIUS, TFT_ORANGE);
427     tft.drawCircle(bombCenterX, bombCenterY, BOMB_RADIUS - 10, TFT_RED);
428
429     bool killedAny = false;
430     for (int i = 0; i < MAX_ENEMIES; i++) {
431         if (enemies[i].active) {
432             float d = sqrt(pow(enemies[i].x - playerX, 2) + pow(enemies[i].y - playerY, 2));
433             if (d <= BOMB_RADIUS) {
434                 enemies[i].active = false;
435                 tft.fillRect((int)enemies[i].x, (int)enemies[i].y, ENEMY_SIZE, ENEMY_SIZE, TFT_BLACK);
436                 killCount++;
437                 spawnGem(enemies[i].x, enemies[i].y);
438                 killedAny = true;
439             }
440         }
441     }
442     if (killedAny) playKillSound();
443 }
444
```

3. Emergency Bomb skill visual & cleanup (ส่วนสกิลระเบิดทำลายล้าง)

- หน้าที่: ตรวจสอบการกดปุ่ม Skill Button (GPIO 25) เมื่อเกจูลดจนพร้อมใช้งาน (BOMB_COOLDOWN) เพื่อปล่อยสกิลระเบิดทำลายศัตรูทั้งหมดในระยะรัศมีระเบิด (BOMB_RADIUS) พร้อมวาดวงกลมเอฟเฟกต์สีส้ม/แดง และเล่นเสียงเอฟเฟกต์ระเบิด

```
445 // 4. Enemy movement & collision
446 for (int i = 0; i < MAX_ENEMIES; i++) {
447     if (!enemies[i].active) {
448         if (random(0, 30) == 0) spawnEnemy(i);
449         continue;
450     }
451
452     enemies[i].oldX = enemies[i].x;
453     enemies[i].oldY = enemies[i].y;
454
455     if (enemies[i].x < playerX) enemies[i].x += enemies[i].speed;
456     else enemies[i].x -= enemies[i].speed;
457
458     if (enemies[i].y < playerY) enemies[i].y += enemies[i].speed;
459     else enemies[i].y -= enemies[i].speed;
460
461     float distToPlayer = sqrt(pow(enemies[i].x - playerX, 2) + pow(enemies[i].y - playerY, 2));
462     if (distToPlayer < 8.0) {
463         enemies[i].active = false;
464         tft.fillRect((int)enemies[i].oldX, (int)enemies[i].oldY, ENEMY_SIZE, ENEMY_SIZE, TFT_BLACK);
465         playerHP--;
466         updateHealthLEDs(playerHP);
467         playHitSound();
468
469         if (playerHP <= 0) {
470             if (survivalTimeSec > bestSurvivalTime) {
471                 bestSurvivalTime = survivalTimeSec;
472             }
473             currentState = STATE_GAMEOVER;
474             stateEnterTime = millis();
475             gameOverScreenDrawn = false;
476             tft.fillRect(TFT_BLACK);
477             return;
478         }
479     }
480 }
481
```

4. Enemy movement & collision (ส่วนการเคลื่อนที่ของศัตรูและการชน)

- หน้าที่: คำนวณการเคลื่อนที่ของศัตรูตามเวลา และคำนวณให้ศัตรูวิ่งเข้าหาตำแหน่งของผู้เล่นตลอดเวลา พร้อมตรวจสอบการชน (Collision) ระหว่างศัตรูกับผู้เล่น หากชนกันจะลดค่า HP, อัปเดตไฟ LED และตั้งเปลี่ยนไปยังหน้า STATE_GAMEOVER เมื่อ HP หมด

Game Logic & Mechanics Module (ส่วนคำนวณเกมและสกิล)

```
482 // 5. Gem collection
483 for (int i = 0; i < MAX_GEMS; i++) {
484     if (gems[i].active) {
485         float d = sqrt(pow(gems[i].x - playerX, 2) + pow(gems[i].y - playerY, 2));
486         if (d < 12.0) {
487             gems[i].active = false;
488             tft.fillRect((int)gems[i].x - 3, (int)gems[i].y - 3, 8, 8, TFT_BLACK);
489             playerXP++;
490             playGemBeep();
491
492             if (playerXP >= xpToNextLevel) {
493                 playerLevel++;
494                 playerXP = 0;
495                 xpToNextLevel += 3;
496
497                 if (playerLevel % 5 == 0) {
498                     hpBonusAwarded = true;
499                     if (playerHP < 3) {
500                         playerHP++;
501                         updateHealthLEDs(playerHP);
502                     }
503                 } else {
504                     hpBonusAwarded = false;
505                 }
506
507                 playLevelFanfare();
508                 stateEnterTime = millis();
509                 levelUpScreenDrawn = false;
510                 lastMenuSelection = -1;
511                 currentState = STATE_LEVELUP;
512                 return;
513             }
514         }
515     }
516 }
517 }
```

5. Gem collection (ส่วนการเก็บไอเทมและการเลเวลอัพ)
- หน้าที่: ตรวจสอบระยะห่างระหว่างผู้เล่นกับ Gem (พลิกอัปเกรด) บนสนาม เมื่อเดินไปชนจะเก็บ Gem เล่นเสียงเอฟเฟกต์ และเพิ่มค่า XP เมื่อสะสม XP ครบตามกำหนด จะคำนวณโบนัส HP และสั่งเปลี่ยนสถานะไปหน้าอัปเกรดตัวละคร (STATE_LEVELUP)

```
// 6. Partial redraws
if ((int)oldPlayerX != (int)playerX || (int)oldPlayerY != (int)playerY) {
    tft.fillRect((int)oldPlayerX, (int)oldPlayerY, PLAYER_SIZE, PLAYER_SIZE, TFT_BLACK);
}
tft.fillRect((int)playerX, (int)playerY, PLAYER_SIZE, PLAYER_SIZE, TFT_CYAN);

for (int i = 0; i < MAX_ENEMIES; i++) {
    if (enemies[i].active) {
        if ((int)enemies[i].oldX != (int)enemies[i].x || (int)enemies[i].oldY != (int)enemies[i].y) {
            tft.fillRect((int)enemies[i].oldX, (int)enemies[i].oldY, ENEMY_SIZE, ENEMY_SIZE, TFT_BLACK);
        }
        tft.fillRect((int)enemies[i].x, (int)enemies[i].y, ENEMY_SIZE, ENEMY_SIZE, TFT_RED);
    }
}

for (int i = 0; i < MAX_GEMS; i++) {
    if (gems[i].active) {
        tft.fillRect((int)gems[i].x, (int)gems[i].y - 2, 2, 6, TFT_GREEN);
        tft.fillRect((int)gems[i].x - 2, (int)gems[i].y, 6, 2, TFT_GREEN);
    }
}

// HUD: XP Bar & Status text
int xpWidth = map(playerXP, 0, xpToNextLevel, 0, SCREEN_W - 8);
tft.fillRect(4, 4, xpWidth, 4, TFT_GREEN);
tft.fillRect(4 + xpWidth, 4, (SCREEN_W - 8) - xpWidth, 4, TFT_DARKGREY);

static unsigned long lastHudTime = 0;
if (millis() - lastHudTime > 300) {
    lastHudTime = millis();
    tft.fillRect(10, 12, 210, 16, TFT_BLACK);
    tft.setTextColor(TFT_WHITE, TFT_BLACK);
    tft.setTextSize(2);
    tft.setCursor(10, 12);
    tft.printf("L%d %02lu:%02lu", playerLevel, survivalTimeSec / 60, survivalTimeSec % 60);

    // Clears X: 228 to 312, leaving 8px margin from right edge
    tft.fillRect(228, 12, 84, 16, TFT_BLACK);
    if (millis() - lastBombTime >= BOMB_COOLDOWN) {
        tft.setTextColor(TFT_YELLOW, TFT_BLACK);
        tft.setCursor(232, 12); // Fits 6 chars at size 2 (72px wide: 232 to 304)
        tft.print("[BOMB]");
    }
}
}
```

6. Partial redraws & HUD (ส่วนการวาดกราฟิกซ่อมแซมและการแสดงผลหน้าจอ)
- หน้าที่: ลบและวาดตำแหน่งใหม่เฉพาะจุดที่มีการเคลื่อนที่ (Partial Redraw) ทั้งผู้เล่น ศัตรู และ Gem เพื่อให้เกมรันได้อย่างลื่นไหลไม่กระตุก พร้อมวาดหลอดแถบพลังประสบการณ์ (XP Bar), ตัวเลขเลเวล, นาฬิกาชีวิต และสถานะความพร้อมของสกิล [BOMB] บนส่วนหัวของหน้าจอ (HUD)

Output Feedback Driver Module (ส่วนขับเคลื่อนฮาร์ดแวร์เอาต์พุต)

```
121 void updateHealthLEDs(int hp) {  
122     digitalWrite(PIN_LED_GRN, hp >= 3 ? HIGH : LOW);  
123     digitalWrite(PIN_LED_YEL, hp >= 2 ? HIGH : LOW);  
124     digitalWrite(PIN_LED_RED, hp >= 1 ? HIGH : LOW);  
125 }  
126
```

updateHealthLEDs(hp):

- ขับสัญญาณ **Digital Output** สั่งงานไฟ **Status LEDs 3** สีภายนอก ให้เปลี่ยนสีเตือนตามระดับ **HP** คงเหลือ: สีเขียว (**GPIO 16**) ติดเมื่อ **HP >= 3**, สีเหลือง (**GPIO 17**) ติดเมื่อ **HP >= 2**, สีแดง (**GPIO 5**) ติดเมื่อ **HP >= 1**

```
127 // Sound Functions  
128 void playSfx(int freq, int durationMs) {  
129     tone(PIN_BUZZER, freq);  
130     buzzerOffTime = millis() + durationMs;  
131     isBuzzerSounding = true;  
132 }  
133  
134 void playGemBeep()    { playSfx(2400, 45); }  
135 void playHitSound()   { playSfx(160, 120); }  
136 void playKillSound()  { playSfx(850, 35); }  
137 void playSelectSound() { playSfx(1400, 40); }  
138 void playLevelFanfare(){ playSfx(3000, 180); }  
139  
140 void updateBuzzer() {  
141     if (isBuzzerSounding && millis() >= buzzerOffTime) {  
142         noTone(PIN_BUZZER);  
143         digitalWrite(PIN_BUZZER, LOW);  
144         isBuzzerSounding = false;  
145     }  
146 }  
147
```

playSfx() & updateBuzzer():

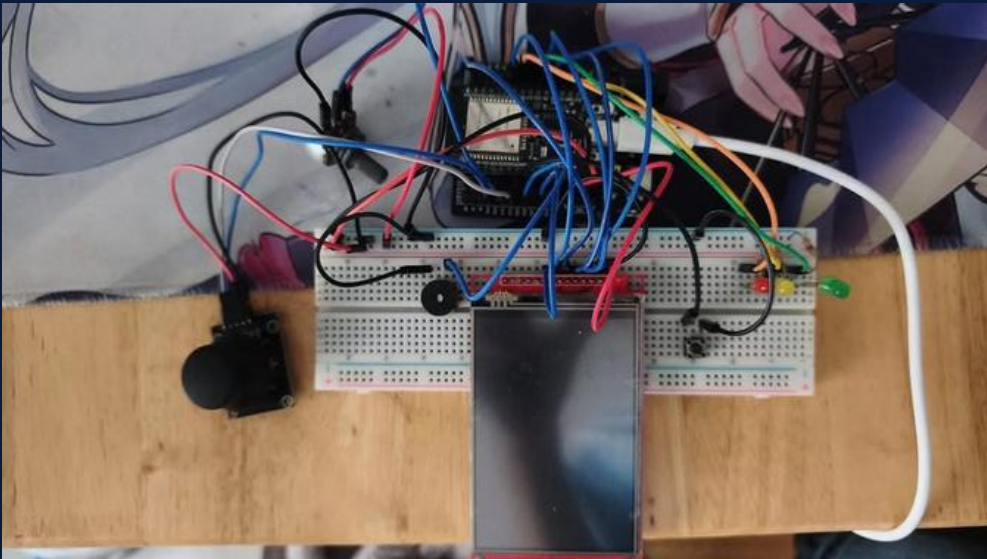
- **playSfx(freq, durationMs)** ตั้งกำเนิดความถี่เสียง **SFX** ออกไปยัง **Passive Buzzer (GPIO 26)** ด้วยฟังก์ชัน **tone()**
- **updateBuzzer()** ตรวจสอบเวลาคับด้วย **millis()** เพื่อตัดสัญญาณด้วย **noTone()** แบบ **Non-blocking** โดยไม่ทำให้จังหวะการรันเกมหยุดชะงัก

Testing and Result (All Cases)

หัวข้อการทดสอบ (Test Case)	เงื่อนไขและวิธีการทดสอบ (Test Condition)	ผลลัพธ์ที่คาดหวัง (Expected Result)	ผลการทดสอบ (Actual Result)
1. Analog Joystick Control	โยกจอยสติ๊กแกน X/Y ออกนอกช่วง Deadband (< 1400 หรือ > 2600)	ตัวละครเคลื่อนที่ตามทิศทางแม่นยำ ไม่ขยับเองเมื่อจอยอยู่นิ่ง	Pass (ผ่าน)
2. Backlight PWM Dimmer	หมุน Potentiometer บนขา GPIO 32 เพื่อปรับค่าแรงดันอนาล็อก	ความสว่างหน้าจอ TFT (GPIO 14) ปรับเพิ่ม/ลดได้สว่างขึ้น/ลง	Pass (ผ่าน)
3. Emergency Skill Bomb	สะสมเกจจนขึ้น [BOMB] แล้วกดปุ่ม PIN_SKILL_BTn (GPIO 25)	เกิดรัศมีระเบิด เคลียร์ศัตรูในระยะ และรีเซ็ตคูลดาวน 4 วินาที	Pass (ผ่าน)
4. External HP LEDs Indicator	ลดค่า playerHP จาก 3 เหลือ 2, 1 และ 0 ตามลำดับ	ไฟ LED เปลี่ยนสีตาม HP: เขียว (>=3) --> เหลือง (>=2) --> แดง (>=1)	Pass (ผ่าน)
5. Non-blocking SFX Buzzer	เกิดเหตุการณ์โจมตี, เก็บ Gem, โดนชน หรือเลเวลอัป	Buzzer (GPIO 26) ส่งเสียง SFX โดยที่เฟรมเรตเกมไม่กระตุก	Pass (ผ่าน)
6. FSM State Transition	ทดสอบการเปลี่ยนสถานะ TITLE --> PLAYING --> LEVELUP --> GAMEOVER	ระบบสลับหน้าจอ และทำงานตามโลจิกของแต่ละสถานะถูกต้อง	Pass (ผ่าน)

Picture of Your Project (Final Work)

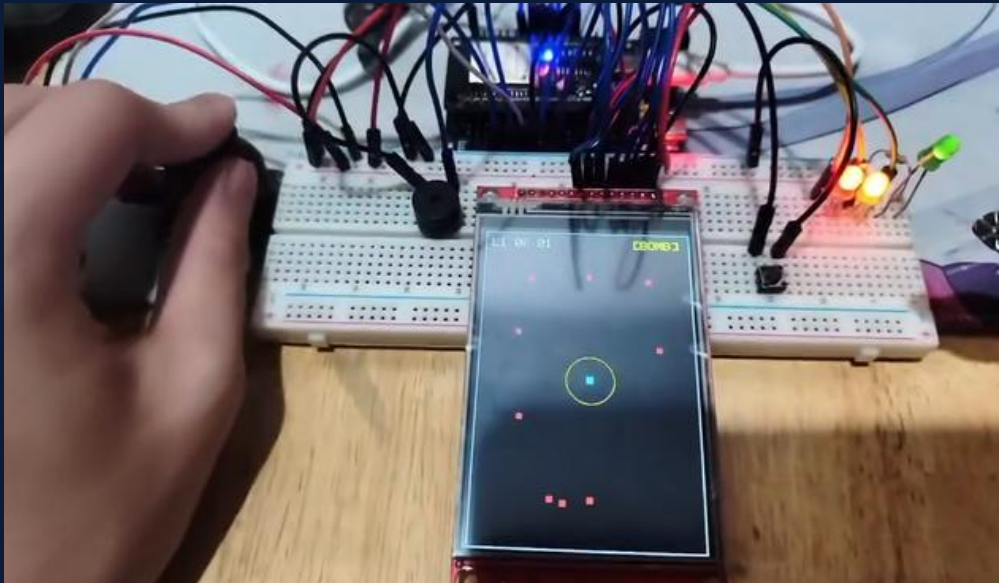
Hardware



Title



Playing



LevelUp



GameOver



LINK to VDO demonstration (final work)

https://drive.google.com/file/d/1wSTmqR_-Sq7AepEVBgjFlw3Xfg82Lu3q/view?usp=drive_link **VDO demonstration (final work)**

Problem and solution

1. ปัญหาสัญญาณอนาล็อกผันผวนจาก Joystick (Joystick Signal Jittering)
 - ปัญหาที่พบ (Problem): ค่าแรงดันอนาล็อกจาก Analog Joystick บนขา ADC (PIN_JOY_X และ PIN_JOY_Y) มีความผันผวนเล็กน้อยตามธรรมชาติของอุปกรณ์ ส่งผลให้ตัวละครขยับเอง หรือเลื่อนเมนูอัตโนมัติเองขณะที่จอยสติ๊กอยู่นิ่ง
 - การแก้ไข (Solution): ออกแบบ Deadband Filtering Algorithm (ช่วงค่า 1400–2600) ในโค้ด โดยกำหนดให้ระบบข้ามการประมวลผลเมื่อค่าอนาล็อกอยู่ในช่วงศูนย์กลาง และจะอัปเดตตำแหน่งก็ต่อเมื่อมีการโยกจอยสติ๊กออกนอกช่วง Deadband เท่านั้น
2. ปัญหาหน้าจอกระตุกจากการวาดภาพกราฟิกใหม่ทั้งหน้าจอ (Screen Redraw Lag)
 - ปัญหาที่พบ (Problem): การใช้คำสั่งลบและวาดหน้าจอใหม่ทั้งหมด (tft.fillScreen()) ทุกๆ เฟรม ขณะมีตัวละคร ศัตรู และ Gem บนหน้าจอจำนวนมาก ทำให้การแสดงผลกระตุก เฟรมเรตตก และหน้าจอกระพริบอย่างเห็นได้ชัด
 - การแก้ไข (Solution): เปลี่ยนมาใช้เทคนิค Partial Redraw (วาดซ่อมเฉพาะจุด) โดยวาดสี่เหลี่ยมสีดำเคลียร์เฉพาะพิกัดเก่า (oldX, oldY) ก่อนจะวาดวัตถุ ณ พิกัดใหม่ ช่วยลดภาระการส่งข้อมูลผ่านพอร์ต SPI และทำให้เกมรันได้ลื่นไหล
3. ปัญหาระบบค้างจากการใช้ delay() ในการจับเวลา (Blocking Code Issues)
 - ปัญหาที่พบ (Problem): การใช้ฟังก์ชัน delay() ในการทำคูลดาวน์สติก หรือการหน่วงเวลาเล่นเสียงเอฟเฟกต์ ทำให้ไมโครคอนโทรลเลอร์หยุดการประมวลผลทั้งหมด ส่งผลให้ปุ่มกดและ Joystick ตอบสนองล่าช้า (Input Lag)
 - การแก้ไข (Solution): สังกายนาโครงสร้างโค้ดทั้งหมดเป็น Non-blocking Logic โดยใช้ฟังก์ชัน millis() เช็ควงต่างเวลา (millis() - lastTime >= INTERVAL) ทั้งระบบจับเวลา FSM, การโจมตีอัตโนมัติ, คูลดาวน์สติกระเบิด และการปิดเสียง Buzzer (updateBuzzer())
4. ปัญหาข้อความ HUD บนหน้าจอแสดงผลทับซ้อนกัน (HUD Display Overlapping)
 - ปัญหาที่พบ (Problem): ข้อความแสดงเวลา สถานะเลเวล และสถานะสติก [BOMB] บริเวณขอบบนของหน้าจอเกิดการวาดตัวเลขซ้อนทับกับตัวเลขเก่าเมื่อค่ามีการเปลี่ยนแปลง
 - การแก้ไข (Solution): ใช้ฟังก์ชัน tft.fillRect() ลบพื้นที่กรอบข้อความเก่าเฉพาะจุดก่อนพิมพ์ตัวเลขใหม่ลงไป พร้อมจัดการขอบเขตพิกัด X/Y ให้พอดีกับความละเอียดจอ 3.8" ในแนวตั้ง (Portrait Mode)

Thank you